

Joins en MariaDB

Juan Carlos Varela Iglesias – jc.foroxinnovacion@gmail.com

Índice

INNER JOIN

Tablas de ejemplo

LEFT JOIN

RIGHT JOIN

FULL JOIN

Tablas de ejemplo

CROSS JOIN

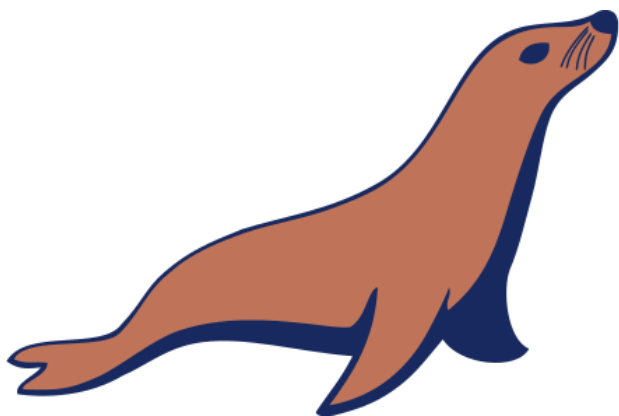
Tablas de ejemplo

SELF JOIN

Tablas de ejemplo

JOIN de JOINS

Tablas de ejemplo



MariaDB

INNER JOIN

También conocido como "join" interno.

Devuelve solo los registros que tienen coincidencias en ambas tablas que se están uniendo. Se seleccionan solo las filas donde los valores de las columnas coinciden.

Tablas de ejemplo

```
CREATE DATABASE mariadb_joins;
USE mariadb_joins;

CREATE TABLE clientes (
  clientes_id INT PRIMARY KEY,
  nombre VARCHAR(50)
);

CREATE TABLE pedidos (
  pedidos_id INT PRIMARY KEY,
  clientes_id INT,
  fecha DATE,
  FOREIGN KEY (clientes_id) REFERENCES clientes(clientes_id)
);

INSERT INTO clientes (clientes_id, nombre)
VALUES (1, "Cliente 1"), (2, "Cliente 2"), (3, "Cliente 3");

INSERT INTO pedidos (pedidos_id, clientes_id, fecha)
VALUES (1,1, "2023-05-10"), (2,2,"2023-06-15");
```

SQL

Obtener los clientes que han realizado pedidos.

```
SELECT c.nombre FROM clientes as c
INNER JOIN pedidos as p
ON c.clientes_id = p.clientes_id;
```

SQL

LEFT JOIN

También conocido como "left outer join".

Devuelve todos los registros de la tabla izquierda (la primera mencionada en la consulta) y los registros coincidentes de la tabla derecha. Si no hay coincidencias en la tabla derecha, se devolverán valores NULL.

Obtener todos los clientes y, si han realizado pedidos, mostrar la fecha del último pedido.

```
SELECT c.nombre, p.fecha FROM clientes AS c
LEFT JOIN pedidos AS p
ON c.clientes_id = p.clientes_id;
```

SQL

Obtener los clientes que han realizado pedidos y mostrar la fecha

```
SELECT * FROM clientes AS c
LEFT JOIN pedidos AS p
ON c.clientes_id = p.clientes_id
WHERE p.clientes_id IS NOT NULL;
```

SQL

RIGHT JOIN

También conocido como "right outer join", es similar al LEFT JOIN, pero invierte el orden de las tablas.

Devuelve todos los registros de la tabla derecha y los registros coincidentes de la tabla izquierda. Si no hay coincidencias en la tabla izquierda, se devolverán valores NULL.

Obtener todos los pedidos y, si están asociados a un cliente, mostrar el nombre del cliente

```
SELECT c.nombre, p.fecha FROM clientes AS c
RIGHT JOIN pedidos AS p
ON c.clientes_id = p.clientes_id;
```

SQL

FULL JOIN

También conocido como "full outer join".

Devuelve todos los registros de ambas tablas, coincidentes y no coincidentes. Si no hay coincidencias, se devolverán valores NULL en las columnas de la tabla opuesta.

Tablas de ejemplo

```
CREATE TABLE empleados (
empleados_id INT PRIMARY KEY,
nombre VARCHAR(50)
);

CREATE TABLE proyectos (
proyectos_id INT PRIMARY KEY,
empleados_id INT,
nombre VARCHAR(50),
FOREIGN KEY (Empleados_id) REFERENCES empleados(Empleados_id)
);

INSERT INTO empleados (Empleados_id, nombre)
VALUES (1, "Empleado 1"), (2, "Empleado 2"), (3, "Empleado 3");

INSERT INTO proyectos (Proyectos_id, empleados_id, nombre)
VALUES
(1,1, "Proyecto 1"),
(2,NULL, "Proyecto 2");
```

SQL

MariaDB NO acepta la sintaxis FULL JOIN



```
SELECT e.nombre, p.nombre FROM empleados as e
FULL JOIN proyectos as p
ON e.empleados_id = p.empleados_id;
```

SQL

Obtener todos los empleados y todos los proyectos, y mostrar las coincidencias si un empleado está asignado a un proyecto

En la siguiente consulta UNION evitaría mostrar resultados duplicados

SQL

```
SELECT * FROM empleados AS e
LEFT JOIN proyectos AS p
ON e.empleados_id = p.empleados_id
UNION
SELECT * FROM empleados AS e
RIGHT JOIN proyectos AS p
ON e.empleados_id = p.empleados_id;
```

En la siguiente consulta, para evitar filas duplicadas, debemos utilizar un patrón anti-join y luego utilizar UNION ALL

SQL

```
SELECT * FROM empleados AS e
LEFT JOIN proyectos AS p
ON e.empleados_id = p.empleados_id
UNION ALL
SELECT * FROM empleados AS e
RIGHT JOIN proyectos AS p
ON e.empleados_id = p.empleados_id
WHERE e.empleados_id IS NULL;
```

Mostrar todos los empleados sin proyecto y todos los proyectos sin empleado asignado

SQL

```
SELECT * FROM empleados AS e
LEFT JOIN proyectos AS p
ON e.empleados_id = p.empleados_id
where p.proyectos_id IS NULL
UNION
SELECT * FROM empleados AS e
RIGHT JOIN proyectos AS p
ON e.empleados_id = p.empleados_id
where e.empleados_id IS NULL;
```

CROSS JOIN

Realiza una combinación cartesiana de las filas de ambas tablas.

Devuelve todas las posibles combinaciones de registros entre las tablas sin ninguna condición de coincidencia.

Tablas de ejemplo

SQL

```
CREATE TABLE productos (
  productos_id INT PRIMARY KEY,
  nombre VARCHAR(50)
);

CREATE TABLE categorias (
  categorias_id INT PRIMARY KEY,
  nombre VARCHAR(50)
);

INSERT INTO productos (productos_id, nombre)
VALUES (1, 'Producto 1'), (2, 'Producto 2'), (3, 'Producto 3');

INSERT INTO categorias (categorias_id, nombre)
VALUES (1, 'Categoría 1'), (2, 'Categoría 2');
```

Obtener todas las combinaciones posibles de productos y categorías

SQL

```
SELECT p.nombre, c.nombre FROM productos as p
CROSS JOIN categorias as c
order by c.nombre;
```

SELF JOIN

Se utiliza para unir una tabla consigo misma. Se utiliza para encontrar relaciones o comparaciones entre filas de la misma tabla.

Tablas de ejemplo

```
CREATE TABLE gerentes (  
  gerentes_id INT PRIMARY KEY,  
  nombre VARCHAR(50)  
);  
  
CREATE TABLE empleados2 (  
  empleados2_id INT PRIMARY KEY,  
  nombre VARCHAR(50),  
  gerentes_id INT,  
  FOREIGN KEY (gerentes_id) REFERENCES gerentes(gerentes_id)  
);  
  
INSERT INTO gerentes (gerentes_id, nombre)  
VALUES (1, "Gerente 1"), (2, "Gerente 2"), (3, "Gerente 3");  
  
INSERT INTO empleados2 (empleados2_id, nombre, gerentes_id)  
VALUES  
(1, "empleados2 1", NULL),  
(2, "empleados2 2", 1),  
(3, "empleados2 3", 2),  
(4, "empleados2 4", 2),  
(5, "empleados2 5", NULL);
```

SQL

Obtener los nombres de los empleados y los nombres de sus respectivos gerentes

```
SELECT e1.empleados2_id AS "ID Empleado", e2.nombre AS "nombre", e1.gerentes_id  
FROM empleados2 AS e1  
JOIN empleados2 AS e2  
#on e1.gerentes_id = e2.gerentes_id  
#on e1.empleados2_id = e2.empleados2_id  
#on e1.empleados2_id = e2.gerentes_id  
on e1.gerentes_id = e2.empleados2_id # <-- Respuesta
```

SQL

JOIN de JOINS

Es posible realizar un "join" entre los resultados de otros "joins", para ello se pueden utilizar subconsultas o vistas derivadas.

Tablas de ejemplo

```
CREATE TABLE clientes_x (  
  clientes_id INT PRIMARY KEY,  
  nombre VARCHAR(50)  
);  
  
CREATE TABLE productos_x (  
  productos_id INT PRIMARY KEY,  
  producto VARCHAR(50)  
);  
  
CREATE TABLE pedidos_x (  
  pedidos_id INT PRIMARY KEY,  
  clientes_id INT,  
  productos_id INT  
);  
  
INSERT INTO clientes_x (clientes_id, nombre)  
VALUES (1, 'Juan'), (2, 'María'), (3, 'Pedro');  
  
INSERT INTO productos_x (productos_id, producto)  
VALUES (101, 'Laptop'), (102, 'Teléfono'), (103, 'Tableta');  
  
INSERT INTO pedidos_x (pedidos_id, clientes_id, productos_id)  
VALUES (1, 1, 101), (2, 2, 102), (3, 1, 103), (4, 3, 101), (5, 2, 103);
```

SQL

Para obtener los clientes que han realizado pedidos de ciertos productos.

- Primero, realizar un "join" entre las tablas "clientes" y "pedidos" para obtener los clientes y sus pedidos.
- Segundo, realizar otro "join" entre los resultados anteriores y la tabla "productos" para obtener los productos asociados a esos pedidos.

```
SELECT c.pedidos_id as "N° Pedido", c.nombre AS "Cliente", p.producto AS "Producto"  
FROM (SELECT clientes_x.clientes_id, clientes_x.nombre, pedidos_x.pedidos_id  
      FROM clientes_x  
      INNER JOIN pedidos_x  
      ON clientes_x.clientes_id = pedidos_x.clientes_id) AS c  
INNER JOIN (SELECT pedidos_x.pedidos_id, productos_x.producto  
            FROM pedidos_x  
            INNER JOIN productos_x  
            ON pedidos_x.productos_id = productos_x.productos_id) AS p  
ON c.pedidos_id = p.pedidos_id;
```

SQL

Otra forma:

```
SELECT c.pedidos_id as "N° de Pedido", c.nombre AS "Cliente", p.producto AS "Producto"  
FROM (SELECT cli.nombre, ped.productos_id, ped.pedidos_id  
      FROM clientes_x AS cli  
      INNER JOIN pedidos_x AS ped  
      ON cli.clientes_id = ped.clientes_id) AS c  
INNER JOIN productos_x AS p  
ON c.productos_id = p.productos_id  
ORDER BY c.nombre;
```

SQL

Last updated 2025-05-29 08:08:08 +0200